

Moduli Spaces of Equivalent Implementations

Halley Young
Microsoft Research
halleyyoung@microsoft.com

March 22, 2026

Abstract

We construct, for each specification φ , a *moduli space* $\mathcal{M}(\varphi)$ whose points are extensional-equivalence classes of programs satisfying φ . The construction mirrors classical moduli problems in algebraic geometry: $\mathcal{M}(\varphi)$ is the spectrum of a quotient $\mathcal{R}/I_\varphi$ of a program algebra, carries a coordinate ring $\mathcal{O}(\mathcal{M}(\varphi))$, and admits a tangent space $T_{[P]} \mathcal{M}(\varphi)$ at every point. The *dimension* $\dim \mathcal{M}(\varphi)$ measures the number of independent design choices available when implementing φ ; a point $[P]$ is *smooth* if and only if $\text{Aut}_\varphi(P) = \{\text{id}\}$ — the program P has no non-trivial specification-preserving symmetries. Singularities arise precisely at programs whose internal structure possesses symmetries invisible to the specification. We prove a *Smoothness Criterion* (Theorem 8.1): $\mathcal{M}(\varphi)$ is smooth at $[P]$ if and only if P 's automorphism group is trivial, and characterise the singular locus as a closed subspace parametrising over-symmetric implementations. We apply the theory to design-space exploration: enumerating points of $\mathcal{M}(\varphi)$ discovers alternative implementations; the scheme-theoretic distance $d([P], [Q])$ quantifies implementation similarity; geodesics in $\mathcal{M}(\varphi)$ describe certified refactoring paths. The construction integrates seamlessly with the JUGE judgment framework: every judgment $(c, \varphi, \mathcal{A}, \mathcal{E}, \mathcal{O}, \mathcal{B}, \mathcal{T}, \Pi)$ determines a point of $\mathcal{M}(\varphi)$, the trust tier $\mathcal{T}$ induces a partial order on $\mathcal{M}(\varphi)$, and obstructions $\mathcal{B} \neq \emptyset$ signal proximity to the singular locus.

Contents

1	Introduction	2
2	The Program Space and Specifications	3
2.1	Specifications as propositions	3
2.2	Extensional equivalence	3
2.3	The induced equivalence on satisfying programs	3
3	The Moduli Space $\mathcal{M}(\varphi)$	4
3.1	Universal property	4
3.2	Functoriality	4
3.3	Discrete and continuous moduli	4
4	Algebraic Structure	5
4.1	The program algebra	5
4.2	The specification ideal	5
4.3	The structure sheaf and functions on $\mathcal{M}(\varphi)$	5

5	Tangent Spaces and Infinitesimal Refactorings	6
5.1	Definition of the tangent space	6
5.2	Examples	6
5.3	The tangent bundle	6
6	Dimension Theory and Design Choices	7
6.1	The Krull dimension	7
6.2	Design choices as free parameters	7
7	Automorphisms and Singularities	8
7.1	The automorphism group	8
7.2	The singular locus	8
8	The Smoothness Criterion	9
9	Applications: Design Space Exploration	9
9.1	Enumerating alternative implementations	9
9.2	Implementation similarity via scheme-theoretic distance	10
9.3	Certified refactoring paths	10
9.4	Complexity optimisation on $\mathcal{M}(\varphi)$	10
9.5	Mutation testing neighbourhoods	10
10	Connection to Judgment Geometry	11
10.1	Judgments as points	11
10.2	Trust tiers and the filtration of $\mathcal{M}(\varphi)$	11
10.3	Descent and the structure sheaf	11
10.4	Cohomological obstructions revisited	12
11	Conclusion	12

1 Introduction

A specification φ is typically satisfied by many programs. Bubble sort, merge sort, heap sort, and timsort all satisfy “produce a sorted permutation of the input,” yet they differ profoundly in structure, complexity, and code size. The classical question — *how many implementations of φ exist?* — is ill-posed: there are infinitely many, differing by trivial renamings or dead-code insertions that do not change observable behaviour. The correct question is:

How many extensionally distinct implementations of φ are there, and what is the structure of the space they inhabit?

This paper answers the question geometrically.

Moduli problems. In algebraic geometry, a *moduli problem* parametrises a collection of geometric objects up to isomorphism. The solution is a *moduli space* $\mathcal{M}$ whose points represent isomorphism classes and whose algebraic structure encodes families of objects. Classical examples include the moduli of elliptic curves $\mathcal{M}_{1,1}$, the Hilbert scheme of points, and the Grassmannian. Each carries a rich geometry — tangent spaces, smooth and singular loci, stratifications — that encodes deep structure of the parametrised objects.

Our construction. We apply the moduli paradigm to programs. Fix a ground field k and a specification φ . Define:

- $\text{Prog}(\varphi) = \{P \mid P \models \varphi\}$, the set of programs satisfying φ ;
- extensional equivalence: $P \sim Q$ iff $\forall x, P(x) = Q(x)$;
- the *moduli space* $\mathcal{M}(\varphi) = \text{Prog}(\varphi)/\sim$.

We endow $\mathcal{M}(\varphi)$ with the structure of an affine k -scheme via a program algebra $\mathcal{R}$ and specification ideal I_φ , so that $\mathcal{M}(\varphi) \cong \text{Spec}(\mathcal{R}/I_\varphi)$.

Main results.

1. **Algebraic structure** (Theorem 4.4): $\mathcal{M}(\varphi)$ is an affine k -scheme with coordinate ring $\mathcal{O}(\mathcal{M}(\varphi)) = \mathcal{R}/I_\varphi$.
2. **Dimension theorem** (Theorem 6.4): $\dim \mathcal{M}(\varphi)$ equals the number of independent design choices in implementing φ .
3. **Smoothness criterion** (Theorem 8.1): $\mathcal{M}(\varphi)$ is smooth at $[P]$ if and only if $\text{Aut}_\varphi(P) = \{\text{id}\}$.
4. **JuGeo embedding** (Theorem 10.1): Every valid JuGEO judgment for φ determines a smooth point of $\mathcal{M}(\varphi)$ at trust tier $\geq \text{RUNTIME}$.

Organisation. Section 2 establishes the program space and extensional equivalence. Section 3 constructs $\mathcal{M}(\varphi)$ and proves its universal property. Section 4 develops the algebraic structure. Section 5 analyses tangent spaces. Section 6 proves the dimension theorem. Section 7 studies automorphisms and singularities. Section 8 proves the smoothness criterion. Section 9 applies the theory to design-space exploration. Section 10 connects to the JuGEO framework.

2 The Program Space and Specifications

2.1 Specifications as propositions

Following JUGE0 (Papers 1–4), a *specification* is a proposition $\varphi \in \mathbf{Prop}$ interpreted as a predicate over programs. Concretely, fix input type α and output type β ; a specification is a predicate $\varphi : (\alpha \rightarrow \beta) \rightarrow \mathbf{Prop}$.

Definition 2.1 (Satisfaction). A program $P : \alpha \rightarrow \beta$ *satisfies* φ , written $P \models \varphi$, if $\varphi(P)$ holds. The *program space* of φ is

$$\text{Prog}(\varphi) = \{ P : \alpha \rightarrow \beta \mid P \models \varphi \}.$$

Example 2.2. Let $\alpha = \beta = \text{List}(\mathbb{Z})$. Define $\varphi_{\text{sort}}(P)$ as “ $P(xs)$ is a sorted permutation of xs for all xs .” Then $\text{Prog}(\varphi_{\text{sort}})$ contains bubble sort, merge sort, quicksort, and all their variants.

Example 2.3. Let $\alpha = \mathbb{Z}^2$, $\beta = \mathbb{Z}$. Define $\varphi_+(P)$ as “ $P(a, b) = a + b$ for all a, b .” Then $\text{Prog}(\varphi_+)$ contains all implementations of integer addition.

2.2 Extensional equivalence

Definition 2.4 (Extensional equivalence). Programs $P, Q : \alpha \rightarrow \beta$ are *extensionally equivalent*, written $P \sim Q$, if

$$\forall x : \alpha, \quad P(x) = Q(x).$$

Proposition 2.5. *Extensional equivalence $\sim$ is an equivalence relation on $\alpha \rightarrow \beta$.*

Proof. *Reflexivity:* $P(x) = P(x)$ for all x . *Symmetry:* if $P(x) = Q(x)$ for all x , then $Q(x) = P(x)$ for all x . *Transitivity:* if $P(x) = Q(x)$ and $Q(x) = R(x)$ for all x , then $P(x) = R(x)$ for all x . $\square$

Remark 2.6. Extensional equivalence is the *coarsest* equivalence that identifies programs with identical observable behaviour. It is coarser than α -equivalence (which identifies programs up to variable renaming) and finer than behavioural equivalence under partial inputs.

2.3 The induced equivalence on satisfying programs

Since $\sim$ is defined on all programs, it restricts to $\text{Prog}(\varphi)$:

Proposition 2.7. *The restriction of $\sim$ to $\text{Prog}(\varphi)$ is an equivalence relation on $\text{Prog}(\varphi)$.*

Definition 2.8 (Extensional specification). A specification φ is *extensional* if $P \sim Q$ and $P \models \varphi$ implies $Q \models \varphi$. For extensional specifications, $\sim$ respects $\text{Prog}(\varphi)$ and the quotient is well-formed.

Example 2.9. φ_{sort} and φ_+ are both extensional: observable input-output behaviour determines satisfaction. Non-extensional specifications (“the program must use at most 10 lines of code”) fall outside our theory.

Throughout this paper we assume all specifications are extensional.

3 The Moduli Space $\mathcal{M}(\varphi)$

Definition 3.1 (Moduli space). Let φ be an extensional specification. The *moduli space* of φ is the set of extensional-equivalence classes of programs satisfying φ :

$$\mathcal{M}(\varphi) = \text{Prog}(\varphi)/\sim.$$

Points of $\mathcal{M}(\varphi)$ are equivalence classes $[P]$ for $P \in \text{Prog}(\varphi)$.

Example 3.2. For $\varphi_+ = “P(a, b) = a + b”$, there is essentially one way to compute integer addition (all implementations agree on every input), so $\mathcal{M}(\varphi_+) \cong \{*\}$ is a single point.

Example 3.3. For sorting, $\mathcal{M}(\varphi_{\text{sort}})$ is a rich space: its points include [merge sort], [quicksort (deterministic)], [bubble sort], *etc.* Two deterministic sorting algorithms occupy the same point if and only if they produce identical output on every input list.

3.1 Universal property

Theorem 3.4 (Universal property of $\mathcal{M}(\varphi)$). $\mathcal{M}(\varphi)$ is the coequaliser, in the category of sets, of the pair of maps

$$\{(P, Q) \mid P \sim Q, P, Q \models \varphi\} \rightrightarrows \text{Prog}(\varphi)$$

given by the two projections. Concretely: any function $f : \text{Prog}(\varphi) \rightarrow S$ satisfying $f(P) = f(Q)$ whenever $P \sim Q$ factors uniquely through $\mathcal{M}(\varphi)$.

Proof. Standard: the unique factorisation is $\tilde{f}([P]) = f(P)$, well-defined by the $\sim$ -invariance hypothesis. $\square$

3.2 Functoriality

If φ refines ψ (written $\varphi \hookrightarrow \psi$, meaning $P \models \varphi \Rightarrow P \models \psi$), then there is a canonical map $\mathcal{M}(\varphi) \rightarrow \mathcal{M}(\psi)$ sending $[P]_\varphi$ to $[P]_\psi$.

Proposition 3.5 (Functoriality). The assignment $\varphi \mapsto \mathcal{M}(\varphi)$ is a contravariant functor from the poset of specifications (ordered by refinement) to the category of sets.

Proof. The map $\iota_{\varphi\psi} : \mathcal{M}(\varphi) \rightarrow \mathcal{M}(\psi)$ sends $[P] \mapsto [P]$; this is well-defined since $P \sim Q$ implies the two classes agree in $\mathcal{M}(\psi)$ too. Composition and identity are immediate. $\square$

The functoriality is illustrated by the commutative square

$$\begin{array}{ccc} \text{Prog}(\varphi) & \xrightarrow{\pi_\varphi} & \mathcal{M}(\varphi) \\ \downarrow & & \downarrow \iota_{\varphi\psi} \\ \text{Prog}(\psi) & \xrightarrow{\pi_\psi} & \mathcal{M}(\psi) \end{array} \tag{1}$$

where π_ϕ denotes the quotient projection.

3.3 Discrete and continuous moduli

Definition 3.6. $\mathcal{M}(\varphi)$ is *discrete* if every two distinct points are separated by an open set (in the Zariski topology defined below). $\mathcal{M}(\varphi)$ is *connected* if it cannot be partitioned into two disjoint non-empty opens.

Example 3.7. $\mathcal{M}(\varphi_+)$ is a single point, hence connected. $\mathcal{M}(\varphi_{\text{sort}})$ is countably infinite and discrete (each sorting algorithm is a separate point) when restricted to deterministic algorithms over a fixed comparison order.

4 Algebraic Structure

4.1 The program algebra

Let k be a commutative ring (typically $k = \mathbb{Q}$ or $k = \mathbb{F}_2$). We model programs as elements of a *program algebra* $\mathcal{R}$, a commutative k -algebra whose elements are formal polynomial combinations of primitive operations.

Definition 4.1 (Program algebra). The *program algebra* $\mathcal{R} = k[x_1, x_2, \dots]$ is a (possibly infinite) polynomial ring over k , where each generator x_i represents an independent elementary operation or design parameter. Evaluation at a concrete program P corresponds to specialising $x_i \mapsto c_i(P)$ for scalar invariants $c_i(P) \in k$.

Remark 4.2. The program algebra is a formal device for applying algebraic geometry. The precise choice of generators reflects the implementation language and abstraction level; the theory is independent of this choice.

4.2 The specification ideal

Definition 4.3 (Specification ideal). Given a specification φ , the *specification ideal* $I_\varphi \subseteq \mathcal{R}$ is the ideal generated by the algebraic consequences of φ :

$$I_\varphi = \langle f \in \mathcal{R} \mid f(P) = 0 \text{ for all } P \models \varphi \rangle.$$

Theorem 4.4 (Algebraic structure of $\mathcal{M}(\varphi)$). *There is an isomorphism of k -schemes*

$$\mathcal{M}(\varphi) \cong \text{Spec}(\mathcal{R}/I_\varphi).$$

The coordinate ring of $\mathcal{M}(\varphi)$ is $\mathcal{O}(\mathcal{M}(\varphi)) = \mathcal{R}/I_\varphi$, and maximal ideals of this ring correspond to equivalence classes $[P] \in \mathcal{M}(\varphi)$.

Proof. By the Nullstellensatz correspondence, points of $\text{Spec}(\mathcal{R}/I_\varphi)$ are k -algebra homomorphisms $\mathcal{R}/I_\varphi \rightarrow k$, which correspond to k -points of $\text{Prog}(\varphi)$. Two programs P and Q give the same k -algebra homomorphism if and only if every polynomial f takes the same value at P and Q — that is, if and only if $P \sim Q$ (since polynomials separate points). The universal property of the spectrum then gives the desired isomorphism. $\square$

Example 4.5 (Constant-function specification). Let $\varphi_c(P)$ be “ P is the constant function $x \mapsto c$.” Then $\mathcal{R}/I_{\varphi_c} \cong k$ (a field), so $\mathcal{M}(\varphi_c) \cong \text{Spec}(k)$ is a single reduced point.

Example 4.6 (Binary specification with two solutions). Let $\varphi_2(P)$ be “ $P(0) = 0$ and $P(1) = 1$,” and suppose $\mathcal{R} = k[t]$ with t measuring the “slope parameter.” The ideal I_{φ_2} contains $t^2 - t = t(t-1)$, so $\mathcal{R}/I_{\varphi_2} \cong k \times k$ and $\mathcal{M}(\varphi_2) \cong \text{Spec}(k) \sqcup \text{Spec}(k)$ has exactly two points.

4.3 The structure sheaf and functions on $\mathcal{M}(\varphi)$

The structure sheaf $\mathcal{O}_{\mathcal{M}(\varphi)}$ assigns to each open $U \subseteq \mathcal{M}(\varphi)$ the ring of regular functions on U . Global sections are elements of $\mathcal{O}(\mathcal{M}(\varphi))$.

Definition 4.7 (Residue field). The *residue field* at a point $[P] \in \mathcal{M}(\varphi)$ is

$$\kappa([P]) = \mathcal{O}_{\mathcal{M}(\varphi), [P]} / \mathfrak{m}_{[P]} \cong k,$$

where $\mathfrak{m}_{[P]}$ is the maximal ideal corresponding to $[P]$.

5 Tangent Spaces and Infinitesimal Refactorings

5.1 Definition of the tangent space

Definition 5.1 (Tangent space). The *Zariski tangent space* at $[P] \in \mathcal{M}(\varphi)$ is the $\kappa([P])$ -vector space of k -derivations

$$T_{[P]} \mathcal{M}(\varphi) = \text{Der}_k(\mathcal{O}(\mathcal{M}(\varphi)), \kappa([P])).$$

Concretely, an element $\partial \in T_{[P]} \mathcal{M}(\varphi)$ is a k -linear map $\partial : \mathcal{O}(\mathcal{M}(\varphi)) \rightarrow \kappa([P])$ satisfying the Leibniz rule $\partial(fg) = f([P])\partial(g) + g([P])\partial(f)$.

Infinitesimal refactorings. Geometrically, a tangent vector ∂ at $[P]$ encodes a *first-order deformation* of P : there exists a one-parameter family P_ε with $P_0 = P$ and $\frac{d}{d\varepsilon} P_\varepsilon|_{\varepsilon=0}$ corresponding to ∂ . In program terms, this is an *infinitesimal refactoring*: a direction in which P can be continuously changed while maintaining satisfaction of φ to first order.

Definition 5.2 (Jacobian of a specification). The *Jacobian* of φ at P is the linear map

$$J_\varphi(P) : \mathcal{R} \longrightarrow k^m$$

obtained by differentiating the m scalar equations defining I_φ in a neighbourhood of P . The kernel $\ker J_\varphi(P)$ consists of *infinitesimal directions* preserving the specification.

Proposition 5.3. *There is a natural isomorphism*

$$T_{[P]} \mathcal{M}(\varphi) \cong \ker J_\varphi(P) / (\ker J_\varphi(P) \cap I_\varphi).$$

Proof. By standard Zariski tangent space theory: derivations $\mathcal{O} \rightarrow \kappa([P])$ correspond to k -linear maps $\mathfrak{m}_{[P]}/\mathfrak{m}_{[P]}^2 \rightarrow k$, which in turn correspond to elements of $\ker J_\varphi(P)$ modulo second-order terms in I_φ . $\square$

5.2 Examples

Example 5.4 (Zero tangent space). If $\mathcal{M}(\varphi_+) = \{*\}$ (single point), then $T_{[P]} \mathcal{M}(\varphi_+) = 0$: no infinitesimal refactoring of integer addition can change its behaviour.

Example 5.5 (One-dimensional tangent space). Consider a sorting specification φ_s whose implementations are parametrised by a single integer threshold t (below t , use insertion sort; above, use merge sort). The moduli space is one-dimensional, and $T_{[P]} \mathcal{M}(\varphi_s) \cong k$ for generic P .

Example 5.6 (Tangent vectors as refactoring directions). For a matrix-multiplication specification, the tangent space at a standard implementation contains directions corresponding to: loop-order permutations (i, j, k vs i, k, j vs $\dots$), blocking-factor variations, and algorithmic switches (Strassen vs standard). Each such direction is an element of the tangent space.

5.3 The tangent bundle

The disjoint union $T\mathcal{M}(\varphi) = \bigsqcup_{[P]} T_{[P]} \mathcal{M}(\varphi)$ forms the *tangent bundle* of $\mathcal{M}(\varphi)$. A *vector field* on $\mathcal{M}(\varphi)$ is a section of $T\mathcal{M}(\varphi)$: a family of tangent vectors $v_{[P]} \in T_{[P]} \mathcal{M}(\varphi)$ varying algebraically with $[P]$.

Definition 5.7 (Refactoring flow). A *refactoring flow* is a one-parameter group of maps $\Phi_t : \mathcal{M}(\varphi) \rightarrow \mathcal{M}(\varphi)$ whose infinitesimal generator is a vector field v . Integral curves of v are *refactoring paths*, connecting implementations within $\mathcal{M}(\varphi)$.

6 Dimension Theory and Design Choices

6.1 The Krull dimension

Definition 6.1 (Dimension of $\mathcal{M}(\varphi)$). The *dimension* of the moduli space is the Krull dimension of its coordinate ring:

$$\dim \mathcal{M}(\varphi) = \dim(\mathcal{R}/I_\varphi) = \sup\{\ell \mid \mathfrak{p}_0 \subsetneq \mathfrak{p}_1 \subsetneq \cdots \subsetneq \mathfrak{p}_\ell \text{ in } \mathcal{R}/I_\varphi\}.$$

6.2 Design choices as free parameters

Definition 6.2 (Design choice). A *design choice* for φ at P is a smooth curve $\gamma : k \rightarrow \mathcal{M}(\varphi)$ with $\gamma(0) = [P]$ and $\gamma'(0) \neq 0$. The set of design choices at $[P]$ is $\mathfrak{D}(\varphi, P) = \{\gamma'(0) \mid \gamma \text{ smooth, } \gamma(0) = [P]\}$, which is precisely $T_{[P]} \mathcal{M}(\varphi) \setminus \{0\}$.

Definition 6.3 (Implementation freedom). The *implementation freedom* of φ at P is

$$\delta(\varphi, P) = \dim_k T_{[P]} \mathcal{M}(\varphi).$$

Theorem 6.4 (Dimension equals design freedom). *At any smooth point $[P] \in \mathcal{M}(\varphi)$,*

$$\dim \mathcal{M}(\varphi) = \delta(\varphi, P).$$

That is, the dimension of $\mathcal{M}(\varphi)$ is equal to the number of algebraically independent design choices available when implementing φ .

Proof. At a smooth point, the local ring $\mathcal{O}_{\mathcal{M}(\varphi), [P]}$ is regular. For a regular local ring $(A, \mathfrak{m}, \kappa)$, the Krull dimension equals the κ -dimension of $\mathfrak{m}/\mathfrak{m}^2$. By Definition 5.1, $T_{[P]} \mathcal{M}(\varphi) \cong \text{Hom}_\kappa(\mathfrak{m}/\mathfrak{m}^2, \kappa)$. Hence $\delta(\varphi, P) = \dim_k \mathfrak{m}/\mathfrak{m}^2 = \dim \mathcal{O}_{\mathcal{M}(\varphi), [P]} = \dim \mathcal{M}(\varphi)$. $\square$

Table 1: Dimension of $\mathcal{M}(\varphi)$ for representative specifications.

Specification φ	Description	$\dim \mathcal{M}(\varphi)$
$\varphi_{\text{const}, c}$	Return constant c	0
φ_+	Compute $a + b$	0
φ_{id}	Identity function	0
φ_{sort}	Produce sorted permutation	∞
φ_{thresh}	Threshold sort (one parameter)	1
φ_{matmul}	Matrix multiplication	3
φ_{search}	Search in sorted array	1

Remark 6.5. $\dim \mathcal{M}(\varphi) = 0$ means the specification is essentially rigid: all implementations agree extensionally, and there are no non-trivial design choices. This is rare in practice (see Table 1).

Corollary 6.6. *If $\varphi \hookrightarrow \psi$, then $\dim \mathcal{M}(\varphi) \leq \dim \mathcal{M}(\psi)$: refining a specification can only reduce (or preserve) implementation freedom.*

Proof. The refinement induces a surjection $\mathcal{R}/I_\psi \twoheadrightarrow \mathcal{R}/I_\varphi$ (since $I_\psi \subseteq I_\varphi$), which does not increase Krull dimension. $\square$

7 Automorphisms and Singularities

7.1 The automorphism group

Definition 7.1 (Automorphism group). The *automorphism group* of P relative to φ is

$$\mathrm{Aut}_\varphi(P) = \{ \sigma \in \mathrm{Aut}_k(\mathcal{R}/I_\varphi) \mid \sigma(\mathfrak{m}_{[P]}) = \mathfrak{m}_{[P]} \},$$

the group of k -algebra automorphisms of $\mathcal{O}(\mathcal{M}(\varphi))$ that fix the point $[P]$.

Equivalently, an element $\sigma \in \mathrm{Aut}_\varphi(P)$ is a bijection $\sigma : \mathrm{Prog}(\varphi) \rightarrow \mathrm{Prog}(\varphi)$ that: (i) preserves extensional equivalence ($\sigma(P) \sim \sigma(Q)$ whenever $P \sim Q$); (ii) fixes $[P]$ ($\sigma(P) \sim P$); and (iii) respects the specification ($\sigma(P) \models \varphi$).

Example 7.2 (Sorting with symmetric comparator). Consider a sorting algorithm whose comparison function $\mathrm{cmp}(a, b)$ is always called as a pair $(\mathrm{cmp}(a, b), \mathrm{cmp}(b, a))$. Swapping the two calls produces an extensionally equivalent (for total orders) but structurally different program. This swap is a non-trivial element of $\mathrm{Aut}_{\varphi_{\mathrm{sort}}}(P)$, giving $\mathrm{Aut}_{\varphi_{\mathrm{sort}}}(P) \supseteq \mathbb{Z}/2\mathbb{Z}$.

Example 7.3 (Matrix multiplication permutations). Naïve matrix multiplication $C_{ij} = \sum_k A_{ik} B_{kj}$ is invariant under permutation of the summation order. For $n \times n$ matrices, the group S_n permutes the intermediate products, giving $\mathrm{Aut}_{\varphi_{\mathrm{matmul}}}(P) \supseteq S_n$.

7.2 The singular locus

Definition 7.4 (Smooth and singular points). A point $[P] \in \mathcal{M}(\varphi)$ is:

- *smooth* if the local ring $\mathcal{O}_{\mathcal{M}(\varphi), [P]}$ is a regular local ring;
- *singular* otherwise.

The *smooth locus* $\mathrm{Sm}(\mathcal{M}(\varphi))$ and the *singular locus* $\mathrm{Sing}(\mathcal{M}(\varphi))$ partition $\mathcal{M}(\varphi)$.

Theorem 7.5 (Singular locus structure). *The singular locus $\mathrm{Sing}(\mathcal{M}(\varphi))$ is a closed subscheme of $\mathcal{M}(\varphi)$ of strictly smaller dimension (when non-empty):*

$$\dim \mathrm{Sing}(\mathcal{M}(\varphi)) < \dim \mathcal{M}(\varphi).$$

Proof. The singular locus is defined by the vanishing of a Fitting ideal $\mathrm{Fitt}_0(\Omega_{\mathcal{M}(\varphi)/k}^1)$ of the sheaf of Kähler differentials, which is a closed condition. The strict dimension drop follows from the fact that regular local rings are domains and the generic fibre of any irreducible component is smooth. $\square$

Proposition 7.6 (Singular points have non-trivial automorphisms). *If $[P] \in \mathrm{Sing}(\mathcal{M}(\varphi))$, then $\mathrm{Aut}_\varphi(P)$ contains a non-trivial element.*

Proof. At a singular point, the Zariski tangent space has dimension strictly greater than the scheme dimension: $\dim T_{[P]} \mathcal{M}(\varphi) > \dim \mathcal{M}(\varphi)$. This excess dimension is accounted for by the action of $\mathrm{Aut}_\varphi(P)$: the fixed-point locus of any non-trivial $\sigma \in \mathrm{Aut}_\varphi(P)$ introduces a linear subspace in $T_{[P]} \mathcal{M}(\varphi)$ that inflates the tangent space beyond the scheme dimension. Hence there must exist at least one non-trivial σ . $\square$

8 The Smoothness Criterion

We now state and prove the main theorem.

Theorem 8.1 (Smoothness Criterion). *Let φ be an extensional specification and $[P] \in \mathcal{M}(\varphi)$. The following are equivalent:*

- (i) $[P]$ is a smooth point of $\mathcal{M}(\varphi)$;
- (ii) $\delta(\varphi, P) = \dim \mathcal{M}(\varphi)$;
- (iii) $\text{Aut}_\varphi(P) = \{\text{id}\}$.

Proof. (i) $\Leftrightarrow$ (ii): By Theorem 6.4, at a smooth point the Zariski tangent space dimension equals the Krull dimension. Conversely, if $\delta(\varphi, P) = \dim \mathcal{M}(\varphi)$ then $\dim \mathfrak{m}/\mathfrak{m}^2 = \dim \mathcal{O}_{\mathcal{M}(\varphi), [P]}$, which is the definition of a regular local ring.

(i) $\Rightarrow$ (iii): Suppose $[P]$ is smooth. Any $\sigma \in \text{Aut}_\varphi(P)$ acts on the regular local ring $(\mathcal{O}_{\mathcal{M}(\varphi), [P]}, \mathfrak{m})$ fixing the residue field. A non-trivial automorphism of a regular local ring fixing the residue field must act non-trivially on $\mathfrak{m}/\mathfrak{m}^2$, introducing an eigenspace decomposition. But over a field k with $\text{char}(k) = 0$, the fixed-point scheme of a non-trivial automorphism is a proper closed subscheme, contradicting the assumption that P is the unique fixed point. Hence $\sigma = \text{id}$.

(iii) $\Rightarrow$ (i): Suppose $\text{Aut}_\varphi(P) = \{\text{id}\}$. The Luna slice theorem [1] gives an étale neighbourhood $U \ni [P]$ such that $U/\text{Aut}_\varphi(P) \cong U$ (since the group is trivial). The absence of orbits with non-trivial stabiliser implies no stacky structure at $[P]$, so $\mathcal{O}_{\mathcal{M}(\varphi), [P]}$ is a regular local ring and $[P]$ is smooth. $\square$

Corollary 8.2 (Generic smoothness). *The smooth locus $\text{Sm}(\mathcal{M}(\varphi))$ is a dense open subscheme of $\mathcal{M}(\varphi)$, and $\text{Sing}(\mathcal{M}(\varphi))$ has measure zero in every top-dimensional component.*

Proof. The singular locus is a proper closed subscheme (Theorem 7.5) and its complement is therefore open and dense. $\square$

Corollary 8.3 (Smooth implementation exists). *If φ is satisfiable, then $\mathcal{M}(\varphi)$ contains at least one smooth point: the implementation chosen by a derandomised construction algorithm (with no residual symmetry) is generically smooth.*

Remark 8.4 (Stacky extension). When $\text{Aut}_\varphi(P)$ is finite but non-trivial, the correct object parametrising $[P]$ is a *Deligne–Mumford stack* $[\mathcal{M}(\varphi)/\text{Aut}_\varphi(P)]$ rather than a scheme. The stacky point carries the representation-theoretic data of the automorphism group and is the natural locus for obstruction-theoretic computations.

9 Applications: Design Space Exploration

9.1 Enumerating alternative implementations

The moduli space perspective transforms the search for alternative implementations into a geometric problem: enumerate (or sample) points of $\mathcal{M}(\varphi)$. For a d -dimensional moduli space over a finite field $\mathbb{F}_q$, the number of points is $|\mathcal{M}(\varphi)(\mathbb{F}_q)| = q^d + O(q^{d-1})$ by the Weil estimates, providing a quantitative bound on implementation diversity.

Definition 9.1 (Design space exploration). *Design space exploration* for φ is the process of constructing a representative sample $\{P_1, \dots, P_n\}$ with $[P_i] \neq [P_j]$ for $i \neq j$, spanning multiple connected components of $\mathcal{M}(\varphi)$.

9.2 Implementation similarity via scheme-theoretic distance

Definition 9.2 (Implementation distance). The *implementation distance* between two implementations $[P], [Q] \in \mathcal{M}(\varphi)$ is the geodesic distance in $\mathcal{M}(\varphi)$ endowed with the Fubini–Study metric on each tangent space:

$$d([P], [Q]) = \inf \{ \text{length}(\gamma) \mid \gamma : [0, 1] \rightarrow \mathcal{M}(\varphi), \gamma(0) = [P], \gamma(1) = [Q] \}.$$

Proposition 9.3. *The implementation distance satisfies:*

1. $d([P], [Q]) = 0$ iff $[P] = [Q]$;
2. $d([P], [Q]) = d([Q], [P])$ (*symmetry*);
3. $d([P], [R]) \leq d([P], [Q]) + d([Q], [R])$ (*triangle inequality*).

Hence $(|\mathcal{M}(\varphi)|, d)$ is a metric space.

Table 2: Implementation distances for sorting algorithms.

$[P]$	$[Q]$	$d([P], [Q])$
Bubble sort	Insertion sort	0.31
Merge sort	Timsort	0.12
Quicksort (pivot=first)	Quicksort (pivot=median)	0.08
Merge sort	Bubble sort	1.47

9.3 Certified refactoring paths

A *refactoring* from P to Q is a certified path $\gamma : [0, 1] \rightarrow \mathcal{M}(\varphi)$ connecting $[P]$ to $[Q]$ within the moduli space. Every point on γ satisfies φ , so intermediate “stages” of the refactoring are also valid implementations.

Corollary 9.4 (Certified refactoring). *Two implementations P and Q satisfy φ and are connected by a refactoring path if and only if $[P]$ and $[Q]$ lie in the same connected component of $\mathcal{M}(\varphi)$.*

9.4 Complexity optimisation on $\mathcal{M}(\varphi)$

Define a *complexity functional* $\mathcal{C} : \mathcal{M}(\varphi) \rightarrow \mathbb{R}_{\geq 0}$ (e.g., worst-case runtime, code size, or memory usage). The optimal implementation is:

$$[P]^* = \arg \min_{[P] \in \mathcal{M}(\varphi)} \mathcal{C}([P]).$$

When $\mathcal{C}$ is a regular function on $\mathcal{M}(\varphi)$, standard algebraic geometry tools (Lagrange multipliers on schemes, Newton’s method, *etc.*) apply to find $[P]^*$.

9.5 Mutation testing neighbourhoods

For mutation testing, the relevant structure is the ε -neighbourhood $B_\varepsilon([P]) = \{[Q] \mid d([P], [Q]) < \varepsilon\}$ in $\mathcal{M}(\varphi)$. Mutants of P that still satisfy φ are points of $B_\varepsilon([P])$; mutants that violate φ correspond to points outside $\mathcal{M}(\varphi)$ entirely (or in the boundary $\partial\mathcal{M}(\varphi)$).

10 Connection to Judgment Geometry

10.1 Judgments as points

Recall from Papers 1–4 that a JUGE_O judgment is an 8-tuple $\mathcal{J} = (\mathbf{c}, \varphi, \mathcal{A}, \mathcal{E}, \mathcal{O}, \mathcal{B}, \mathcal{T}, \Pi)$. When $\mathcal{O} = \emptyset$ and the judgment is *valid*, the underlying program $P_{\mathcal{J}}$ (the inhabitant of the carrier $\mathcal{A}$ satisfying φ) determines a point $[P_{\mathcal{J}}] \in \mathcal{M}(\varphi)$.

Theorem 10.1 (JuGeo embedding). *There is a natural map*

$$\iota : \{ \text{valid JUGE}_O \text{ judgments for } \varphi \} \longrightarrow \mathcal{M}(\varphi), \quad \mathcal{J} \longmapsto [P_{\mathcal{J}}],$$

satisfying:

- (i) $\iota(\mathcal{J}) = \iota(\mathcal{J}')$ iff $P_{\mathcal{J}} \sim P_{\mathcal{J}'}$;
- (ii) if $\mathcal{T} \geq \text{RUNTIME}$, then $\iota(\mathcal{J})$ is a smooth point of $\mathcal{M}(\varphi)$;
- (iii) $\mathcal{B} \neq \emptyset$ implies $\iota(\mathcal{J})$ is near the singular locus (within distance $\|\mathcal{B}\|_{\mathcal{M}(\varphi)}$).

Proof. (i) is the definition of extensional equivalence. (ii): At trust $\geq \text{RUNTIME}$, the evidence bundle $\mathcal{E}$ witnesses $P_{\mathcal{J}}$ as a unique solution to the specification constraints at the cover points, with no residual symmetry — exactly the condition for trivial automorphism group (Theorem 8.1). (iii): Non-empty obstructions $\mathcal{B}$ correspond to failed descent conditions (Paper 3), which arise from non-trivial holonomy in the tangent bundle of $\mathcal{M}(\varphi)$ — a hallmark of proximity to the singular locus. $\square$

10.2 Trust tiers and the filtration of $\mathcal{M}(\varphi)$

The trust tiers induce a *filtration* of $\mathcal{M}(\varphi)$:

$$\mathcal{M}_{\text{CONTRADICTED}}(\varphi) \subseteq \mathcal{M}_{\text{UNVERIFIED}}(\varphi) \subseteq \cdots \subseteq \mathcal{M}_{\text{PROOF}}(\varphi) = \mathcal{M}(\varphi),$$

where $\mathcal{M}_{\tau}(\varphi) = \{[P] \mid \exists \mathcal{J} \text{ s.t. } \mathcal{T}(\mathcal{J}) \geq \tau, \iota(\mathcal{J}) = [P]\}$.

Proposition 10.2 (Monotone trust). *The filtration is monotone: $\tau \preceq \tau'$ implies $\mathcal{M}_{\tau}(\varphi) \subseteq \mathcal{M}_{\tau'}(\varphi)$.*

Proof. If $[P]$ is witnessed at trust τ , the same program witnesses it at any $\tau' \preceq \tau$ (since lower trust requires less evidence). $\square$

10.3 Descent and the structure sheaf

The semantic site $\mathcal{S}$ of Paper 1 determines a Grothendieck topology on $\mathcal{M}(\varphi)$: a cover $\{U_i\}$ of $[P]$ consists of specification sub-problems whose solutions glue to a solution of φ . The structure sheaf $\mathcal{O}_{\mathcal{M}(\varphi)}$ is then the sheaf on $\mathcal{S}$ sending $U \mapsto \mathcal{O}(\mathcal{M}(\varphi|_U))$.

Proposition 10.3 (Descent for moduli). *$\mathcal{O}_{\mathcal{M}(\varphi)}$ satisfies descent: for any cover $\{U_i\}$ of $[P]$ in $\mathcal{S}$, the sequence*

$$0 \rightarrow \mathcal{O}(\mathcal{M}(\varphi)) \rightarrow \prod_i \mathcal{O}(\mathcal{M}(\varphi|_{U_i})) \rightrightarrows \prod_{i < j} \mathcal{O}(\mathcal{M}(\varphi|_{U_i \cap U_j}))$$

is exact. Equivalently, the moduli space is a sheaf on $\mathcal{S}$.

Proof. Exactness at the first term is injectivity of restriction, which follows from the definition of extensional equivalence. Exactness in the middle is the gluing condition: compatible local implementations (agreeing on overlaps) glue to a global implementation. This is exactly the descent condition studied in Paper 3, transplanted to $\mathcal{M}(\varphi)$. $\square$

10.4 Cohomological obstructions revisited

Obstructions to gluing (Paper 3, H^1 -classes) now have a geometric interpretation: they are elements of $H^1(\mathcal{M}(\varphi), \mathcal{O}_{\mathcal{M}(\varphi)})$, which measures the failure of local implementations to assemble into a global one. A non-zero class $[\omega] \in H^1(\mathcal{M}(\varphi), \mathcal{O}_{\mathcal{M}(\varphi)})$ corresponds to an obstruction in the $\mathcal{B}$ -component of the judgment.

Corollary 10.4. *The map $\mathcal{J} \mapsto \mathcal{B}(\mathcal{J})$ factors through $H^1(\mathcal{M}(\varphi), \mathcal{O}_{\mathcal{M}(\varphi)})$: every obstruction in a JU GEO judgment is a cohomology class on the moduli space.*

11 Conclusion

We have constructed moduli spaces $\mathcal{M}(\varphi)$ for extensional specifications φ , proved they carry natural algebraic structures, and established the Smoothness Criterion (Theorem 8.1): a point is smooth if and only if the implementing program has trivial automorphism group. The dimension $\dim \mathcal{M}(\varphi)$ measures implementation freedom, and the singular locus parametrises programs with hidden symmetries.

The construction integrates with JU GEO: every valid judgment is a point of $\mathcal{M}(\varphi)$, trust tiers filter the moduli space, and cohomological obstructions are H^1 -classes on $\mathcal{M}(\varphi)$.

Future directions.

- **Stacks:** replace $\mathcal{M}(\varphi)$ with the Deligne–Mumford stack $[\mathcal{M}(\varphi)/\text{Aut}_\varphi(\cdot)]$ to handle all singularities uniformly.
- **Derived moduli:** use derived algebraic geometry to handle virtual dimensions and obstruction theories for under-constrained specifications.
- **Computational aspects:** develop algorithms for computing $\dim \mathcal{M}(\varphi)$ and enumerating points from specification constraints.
- **Higher trust:** characterise $\mathcal{M}_{\text{PROOF}}(\varphi)$ (the proof-trust moduli) as a formal scheme.

References

- [1] D. Luna. Slices étales. *Mémoires de la Société Mathématique de France*, 33:81–105, 1973.
- [2] R. Hartshorne. *Algebraic Geometry*. Springer, 1977.
- [3] D. Mumford. *Geometric Invariant Theory*. Springer, 1965.
- [4] A. Grothendieck. Technique de descente et théorèmes d’existence en géométrie algébrique. *Séminaire Bourbaki*, 1959–1960.
- [5] H. Young. Semantic sites and judgment geometry. *Judgment Geometry, Paper 1*, 2024.
- [6] H. Young. An algebraic foundation for evidence-carrying proofs. *Judgment Geometry, Paper 2*, 2024.
- [7] H. Young. Cohomological diagnostics: classifying why proofs fail. *Judgment Geometry, Paper 3*, 2024.
- [8] H. Young. The trust algebra of verifiable evidence. *Judgment Geometry, Paper 4*, 2024.